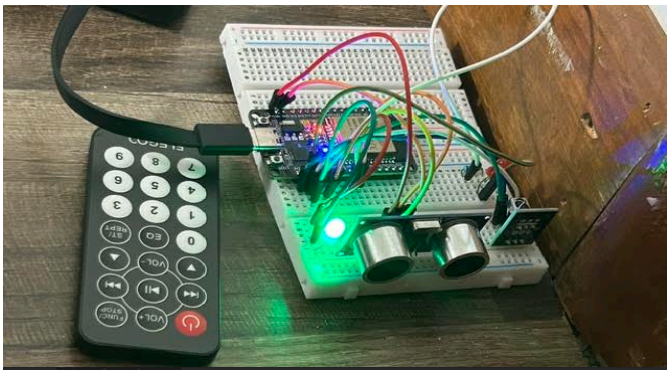
Build A Portable Motion Alert System That Pings Your Phone (ESP32 + HC-SR04)

By: Priyansh Nautiyal



In today’s age of technology, automation is influential in many places: manufacturing, finance, healthcare. It shows up in different areas as well; for instance, there are automated chicken coop doors now, keeping the predators out at night. Which says something: if automation is worth building for a chicken coop, it was only a matter of time before it showed up at the front door of your house.

Home security is where that shift is more obvious. A lock and a watchdog used to be the system, but now it’s motion sensors, door cameras, and app alerts that text you the second something moves. All of which costs money. The average U.S. home security setup costs about $600 before you even add monitoring, and a full custom build can push past $3,000. For a renter, student, or anyone who just wants to know when something crosses one specific spot, that’s a lot of system for a very small need.

That is the area where this project fits: what’s the cheapest, simplest version of “tell me when something passes” you can actually build on your own?

What actually separates this from "a cheaper camera" is that it's built to move with you. One unit, no wiring into walls, and it recalibrates whenever you set it down and turn it on. That one trait quietly opens up who it's useful for.

Renters are the obvious case. Most security systems assume you own the walls you're mounting into; most people renting an apartment don't. Same problem for an Airbnb host, a dorm room, anywhere you're not staying forever.

It's also useful in a quieter way: checking on an elderly parent living alone without putting a camera in their living room, which plenty of people find invasive no matter how good the intention is. A log of hallway activity tells you someone's up and moving without watching them do it.

And yeah, parents want to know when a toddler wanders off. This can do that, and pretty cheaply. But it's not a baby monitor, and it shouldn't be treated like one. This is a breadboard-built awareness tool, not a certified safety device, and that matters more here than almost anywhere else in the paper.

Who is this for?

- Renters, or anyone who can’t drill into walls
- Workshops and garages where you’d like a heads-up if someone wanders in
- Dorms, rentals, storage units, or places you won’t stay forever, because the device is portable

One honest warning: this is a project build, so it shouldn’t be used for anything that people need to rely on for safety. There are actually tested, safe, reliable, and approved products on the market that are made for public safety, and considerations for other aspects to make sure that the product can be relied upon.

Where I started

I began with an Arduino Uno R3 starter kit, the kind with a pile of sensors, LEDs, and jumper wires. The sensing part was easy because the kit has an HC-SR04 ultrasonic distance sensor. The phone part was the problem. The Uno has no WiFi or Bluetooth, so it can't talk to a phone app on its own.

I could have added a WiFi module to the Uno, but then two chips have to talk to each other over a serial link, which means more wiring and more ways to break. An ESP32 board has WiFi built in, programs from the same Arduino IDE, and costs about as much as lunch. So the Uno stayed in the kit, and the ESP32 became the brain. The sensor, resistors, LEDs, and wires all came from the kit.

What Does It Do?

- A green LED remains on while it’s turned on and running, and nothing is happening.
- When the distance in front of the sensor changes, the red LED blinks and your phone gets a notification.
- An IR remote can turn it on or off, and can recalibrate it for wherever you put it.

How Does It Work?

The HC-SR04 sends out a short ultrasonic ping, and it times how long it takes to come back. Sound travels at about 343 meters per second, so that time converts straight into a distance.

When the device starts, and any time you recalibrate, it takes five readings and averages them. That average is the baseline: the distance to whatever normally sits in front of it, like the far wall of a hallway. After that, it keeps measuring. If a reading is different from the baseline by more than a set amount, something changed. The ESP32 blinks the red LED and sends an alert through Blynk, a free platform for connecting small boards to a phone app. It sends at most one alert every 10 seconds, so a person standing in front of it doesn't bury your phone.

The baseline is what makes it portable. You never type in a distance. Put it 40 cm from a wall or 3 meters down a hallway, press recalibrate, and it learns the new spot.

Parts:

Part	**Quantity**	**Description**
ESP32 dev board (30-pin, ESP32-WROOM-32, USB-C, CP2102 chip)	1	The brain, with WiFi built in. Often sold in 3-packs.
HC-SR04 ultrasonic sensor	1	In most Arduino starter kits
Red LED and green LED	1 each	
220 ohm resistors	2	One per LED
1k ohm resistor and 2k ohm resistor	1 each	For the voltage divider. Two 1k resistors in series work as a 2k.
IR receiver module and remote	1 each	Also in most kits.
Breadboard and jumper wires		You need male-to-male, plus a few male-to-female
USB cable that carries data	1	Some cables are charge-only, and then your computer won't see the board
USB power bank	1	To run it away from your computer

Software: the free Arduino IDE, a free Blynk account, and the Blynk app on your phone.

How To Build It?

Step 1: Set up the Arduino IDE for the ESP32

1. In the Arduino IDE, open File > Preferences. In Additional Boards Manager URLs, paste https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

2. Open Tools > Board > Boards Manager, search for esp32, and install the package by Espressif Systems.

3. Open Tools > Board and pick ESP32 Dev Module. Your board won't have its own name in the list, and this generic entry works.

4. Plug in the board and choose its port under Tools > Port.

5. Open Sketch > Include Library > Manage Libraries and install Blynk (by Blynk) and IRremote (by Armin Joachimsmeyer).

If your computer doesn't see the board at all, try a different USB cable first.

Step 2: Make the board blink

Before wiring anything, upload the built-in Blink example (File > Examples > 01.Basics > Blink). Most ESP32 boards have a small LED on GPIO 2, and it should start blinking. A GPIO is just one of the numbered pins you can wire things to.

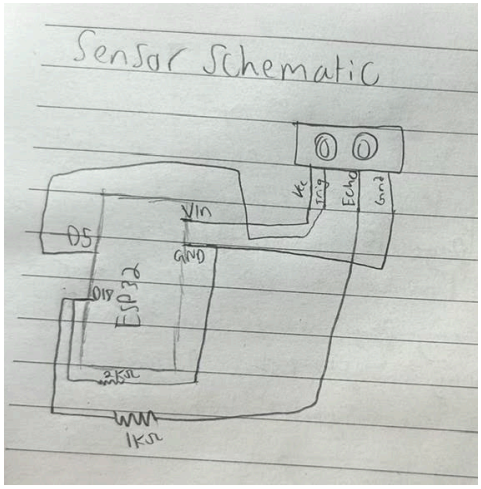
Step 3: Wire the sensor

The HC-SR04 runs on 5V, and its Echo pin sends back a 5V signal. The ESP32's pins are only rated for 3.3V, and feeding 5V into one can damage the chip. The fix is a voltage divider, which is two resistors that split the voltage.

Echo goes through the 1k resistor to GPIO 18. From that same point, the 2k resistor goes to ground. The pin sees 5V x 2k / (1k + 2k), which is about 3.3V. Trig doesn't need a divider. The ESP32's 3.3V output is enough to trigger the sensor.

- HC-SR04 VCC ---> ESP32 VIN(5V)
- HC-SR04 GND ---> GND
- HC-SR04 Trig ---> GPIO 5
- HC-SR04 Echo ---> 1k resistor, then GPIO 18
- GPIO 18 (same point) ---> 2K resistor, then GND

The ESP32 only has a few GND pins, so run one wire from GND to your breadboard's ground rail and connect everything else to that rail. Pin labels vary a little between boards, so check the printing on yours.



Step 4: Test the sensor

Upload this and open Tools -> Serial Monitor. Set the speed in the bottom right to 115200 baud.

```
const int TRIG_PIN = 5;
const int ECHO_PIN = 18;

void setup() {
  Serial.begin(115200);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
}

void loop() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH, 30000);
  float distanceCM = (duration * 0.0343) / 2.0;

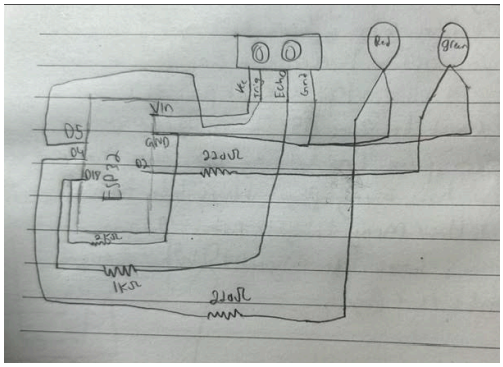
  Serial.print("Distance: ");
  Serial.print(distanceCM);
  Serial.println(" cm");
  delay(500);
}
```

Wave your hand in front of the sensor. The number should drop as your hand gets closer. A reading of 0.00 means no echo came back.

If the Serial Monitor shows a mess of strange characters, the speed setting doesn't match the 115200 baud in the code. Change the dropdown, and it clears up. That happened to me, and it looks scarier than it is.

Step 5: Add the LEDs

The longer leg of an LED is the positive side. Connect it through a 220 ohm resistor to the GPIO pin, and the shorter leg to ground.



- Green LED Long leg ---> 220-ohm resistor, then GPIO 2.
- Green LED short leg ---> ground
- Red LED long leg ---> 220-ohm resistor, then GPIO 4
- Red LED short leg ---> ground

Step 6: What counts as motion?

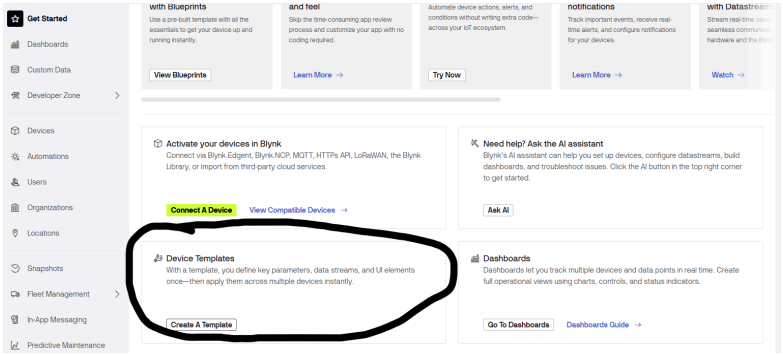
This is where I lost the most time. My first version only fired when something got 15 cm closer than the baseline. On my desk, the baseline was about 17.5 cm, which put the trigger point at roughly 2.5 cm from the sensor. That's basically touching it, so waving my hand around did nothing, and I spent a while convinced something was broken.

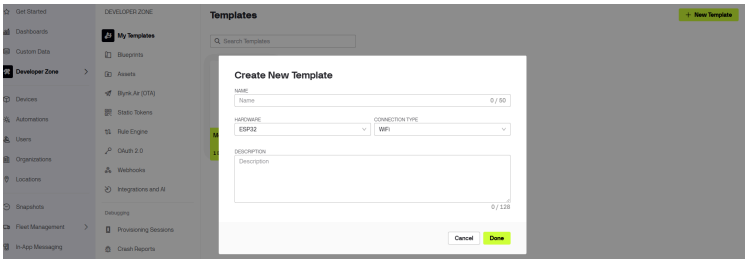
The fix was to stop asking whether something got closer and ask whether the reading changed. If the distance differs from the baseline by more than a threshold in either direction, it fires. That works at any mounting distance. Start at 5 cm and lower it if it isn't sensitive enough.

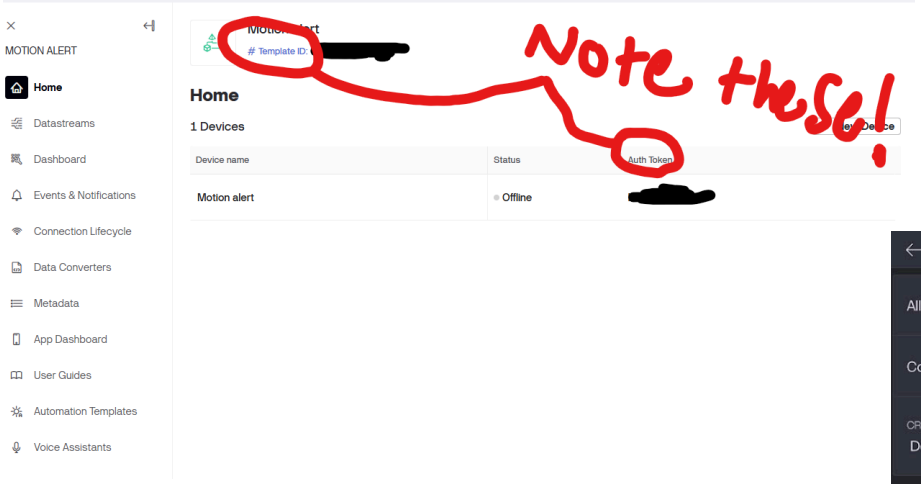
A tip for mounting: point the sensor across the narrowest part of the space, like straight across a doorway, instead of out into an open room. The sensor sees a cone, not a line, so open space gives you false alarms from things that aren't really passing through.

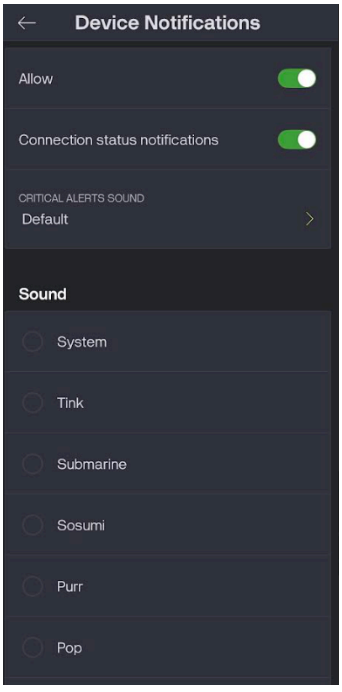
Step 7: Set Up Blynk

1. Make a free account at blynk.io and open the web console.
2. Create a new Template. Pick ESP32 as the hardware and WiFi as the connection.
3. In the template, add an Event with the code motion_detected. It has to match the sketch exactly. Make sure the event is set to send a push notification.
4. Create a Device from your template. Its page shows three values: Template ID, Template Name, and Auth Token.
5. Install the Blynk app on your phone, log in to the same account, and allow notifications.


Get Started
Dashboards
Custom Data
Developer Zone
Devices
Automations
Users
Organizations
Locations
Snapshots
Fleet Management
In-App Messaging
Predictive Maintenance
with Blueprints
Use a pre-built template with all the essentials to get your device up and running instantly.
View Blueprints
and feel
Skip the time-consuming app review process and customize your app with no coding required.
Learn More
Automate device actions, alerts, and conditions without writing extra code—across your IoT ecosystem.
Try Now
notifications
Track important events, receive real-time alerts, and configure notifications for your devices.
Learn More
with Datastreams
Watch
Activate your devices in Blynk
Connect via Blynk.Edgent, Blynk.NCP, MQTT, HTTPs API, LoRaWAN, the Blynk Library, or import from third-party cloud services.
Connect A Device
View Compatible Devices
Need help? Ask the AI assistant
Blynk's AI assistant can help you set up devices, configure datastreams, build dashboards, and troubleshoot issues. Click the AI button in the top right corner to get started.
Ask AI
Device Templates
With a template, you define key parameters, data streams, and UI elements once—then apply them across multiple devices instantly.
Create A Template
Dashboards
Dashboards let you track multiple devices and data points in real time. Create full operational views using charts, controls, and status indicators.
Go To Dashboards
Dashboards Guide


Templates
New Template
Create New Template
NAME
Name
0 / 50
HARDWARE
ESP32
CONNECTION TYPE
WiFi
DESCRIPTION
Description
0 / 128
Cancel
Done


MOTION ALERT
Home
Datastreams
Dashboard
Events & Notifications
Connection Lifecycle
Data Converters
Metadata
App Dashboard
User Guides
Automation Templates
Voice Assistants
Template ID:
Note these!
Home
1 Devices
Device name
Status
Auth Token
Motion alert
Offline


Device Notifications
Allow
Connection status notifications
CRITICAL ALERTS SOUND
Default
Sound
System
Tink
Submarine
Sosumi
Purr
Pop

Step 8: IR Remote Setup

A remote means you can turn the device off or recalibrate it from across the room. Most starter kits include a small IR remote and a three-pin receiver. Mine was labeled G, R, and Y, which are ground, power, and signal.

- G (Ground) ---> GND
- R (Power) ---> 3.3V
- Y (signal) ---> GPIO 15

Every remote sends its own different codes, so first find out what yours sends.

Upload the code below in a new sketch, open the Serial Monitor at 115200 baud, and press the buttons you want to use for power and calibration.

```
#include <IRremote.hpp>

const int IR_PIN = 15;

void setup() {

 Serial.begin(115200);

 IrReceiver.begin(IR_PIN);

 Serial.println("Point the remote at the receiver and press a button...");

}

void loop() {

 if (IrReceiver.decode()) {

  Serial.print("Code: 0x");

  Serial.println(IrReceiver.decodedIRData.decodedRawData, HEX);

  IrReceiver.resume();

 }

}
```

On my remote, the power button printed BA45FF00 and the 0 button printed E916FF00. Yours will differ. Then add these codes to the full sketch given in the next step.

Step 9: The Full Sketch

Replace the blue with your own information:

```
#define BLYNK_TEMPLATE_ID "YOUR_TEMPLATE_ID"
#define BLYNK_TEMPLATE_NAME "YOUR_TEMPLATE_NAME"
#define BLYNK_AUTH_TOKEN "YOUR_AUTH_TOKEN"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <IRremote.hpp>

char ssid[] = "YOUR_WIFI_NAME";
char pass[] = "YOUR_WIFI_PASSWORD";

const int TRIG_PIN = 5;
const int ECHO_PIN = 18;
const int GREEN_LED = 2;
const int RED_LED = 4;
const int IR_PIN = 15;

// IR remote codes
const unsigned long IR_POWER_CODE = IR_POWER_CODE;      // put in YOUR code for power
button
const unsigned long IR_CALIBRATE_CODE = IR_CALIBRATE_CODE;  // put in YOUR code for "0"
button

const float CHANGE_THRESHOLD_CM = 3.0;
const unsigned long NOTIFY_COOLDOWN_MS = 10000;  // don't spam notifications (10 second
cooldown)

bool systemOn = true;
float baselineDistance = 0;
unsigned long lastNotifyTime = 0;

float readDistanceCM() {
  digitalWrite(TRIG_PIN, LOW);
```

```
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH, 30000);
  if (duration == 0) return -1;
  return (duration * 0.0343) / 2.0;
}

void calibrate() {
  Serial.println("Calibrating...");
  float total = 0;
  int validReadings = 0;
  for (int i = 0; i < 5; i++) {
    float d = readDistanceCM();
    if (d > 0) {
      total += d;
      validReadings++;
    }
    delay(100);
  }
  if (validReadings > 0) {
    baselineDistance = total / validReadings;
    Serial.print("New baseline distance: ");
    Serial.print(baselineDistance);
    Serial.println(" cm");
  } else {
    Serial.println("Calibration failed - no valid readings. Check wiring.");
  }
}

void togglePower() {
  systemOn = !systemOn;
  Serial.println(systemOn ? "System turned ON" : "System turned OFF");
  digitalWrite(GREEN_LED, systemOn ? HIGH : LOW);
  digitalWrite(RED_LED, LOW);
}

void setup() {
  Serial.begin(115200);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(GREEN_LED, OUTPUT);
```

```
  pinMode(RED_LED, OUTPUT);

  IrReceiver.begin(IR_PIN);

  Serial.println("Connecting to WiFi and Blynk...");
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
  Serial.println("Connected!");

  calibrate();
  digitalWrite(GREEN_LED, HIGH);
}

void loop() {
  Blynk.run();

  // IR remote
  if (IrReceiver.decode()) {
    unsigned long code = IrReceiver.decodedIRData.decodedRawData;

    if (code == IR_POWER_CODE) {
      togglePower();
    } else if (code == IR_CALIBRATE_CODE) {
      calibrate();
    }

    IrReceiver.resume();
  }

  // Motion check
  if (systemOn) {
    float distance = readDistanceCM();

    if (baselineDistance > 0 && distance > 0 && abs(distance - baselineDistance) >
CHANGE_THRESHOLD_CM) {
      Serial.print("MOTION DETECTED! Distance: ");
      Serial.print(distance);
      Serial.println(" cm");

      unsigned long now = millis();
      if (now - lastNotifyTime > NOTIFY_COOLDOWN_MS) {
        Blynk.logEvent("motion_detected", "Motion Has Been Detected!");
        lastNotifyTime = now;
      }
```

```
    for (int i = 0; i < 4; i++) {
      digitalWrite(GREEN_LED, LOW);
      digitalWrite(RED_LED, HIGH);
      delay(150);
      digitalWrite(RED_LED, LOW);
      delay(150);
    }
    digitalWrite(GREEN_LED, HIGH);
  }
 }

 delay(100);
}
```

Step 10: Power It!

For testing, the USB cable from your computer is fine. To run it somewhere else, use a USB power bank, since it already puts out a clean 5V.

My first try was a 9V battery through a small power module. The board lit up, but the system didn't work properly. A 9V battery is built for things like smoke detectors that sip power slowly. The ESP32 pulls short, sharp bursts of current when WiFi transmits, and a 9V battery can't deliver those without its voltage sagging, which makes the board glitch or reset. I'd skip the 9V route entirely.

Troubleshooting:

Wrong boot mode detected (0x13) on upload	Hold the BOOT button on your ESP32 while the IDE says “Connecting…”
Garbled text in the serial monitor	Set the serial monitor speed to 115200 baud
Computer doesn’t see the board	Use a Data USB cable
Distance reads 0.00 or -1	Check the Trig and Echo wires and the divider. The sensor can't read objects closer than a couple of centimeters.
Readings work, but it never triggers.	Print the baseline. Make sure the sensor points at open space, not at a desk or a wire, and try a lower threshold.
No phone notification	Check your WiFi details, make sure the event code is spelled exactly motion_detected, and make sure notifications are on in the phone app.

	Remember the 10-second cooldown.
IR receiver prints nothing	Reseat the signal wire first. If the signal pin still sits at a constant 1 while you press buttons, try moving it to GPIO 27.
Board powers on but acts strange on a battery	Don’t use a 9V battery. Use a USB power bank when away from the computer.

What It Can and Can’t Do:

This device is a distance-change detector, and not a true motion sensor. It reacts when the distance in front of it changes, inside a fairly narrow cone, out to a few meters.

• Soft or angled surfaces like curtains and clothing bounce sound poorly, so some things pass by less reliably.
• Fans, pets, and vibration can cause false alarms.
• It needs WiFi. If the network is down when it boots, the sketch waits until it connects.
• It isn't weatherproof, so keep it indoors.
• The remote only works with a line of sight to the receiver.
• It isn't a certified safety device. Use it for awareness, not for anything where a missed alert could hurt someone.

Get The Files:

Code in the repository: https://github.com/PriyanshNautiyal/Motion_Detector

- Firmware: the full sketch, the IR reader sketch

License: MIT (See the license on GitHub).

Workes Cited

"The Benefits of an Automatic Door for Chicken Coops." *Nestera*, nestera.it/en/blogs/blog/the-benefits-of-an-automatic-door-for-chicken-coops. Accessed 3 Sept. 2026.

Blynk Technologies. *blynk-library*. GitHub, github.com/blynkkk/blynk-library. Accessed 3 Sept. 2026.

ElecFreaks. "Ultrasonic Ranging Module HC-SR04." *Radiolocman*, www.radiolocman.com/datasheet/pdf.html?di=168043. Accessed 3 Sept. 2026.

ESP32 DevKit C V2 Pin Guide. GitHub, github.com/bitbank2/Thermal_Printer/files/10664674/ESP-32.Dev.Kit.C.V2_EN.pdf. Accessed 3 Sept. 2026.

ESP32-WROOM-32 Datasheet. Version 3.4, Espressif Systems, www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf. Accessed 3 Sept. 2026.

"Events Tutorial." *Blynk Documentation*, Blynk, docs.blynk.io/en/getting-started/events-tutorial. Accessed 3 Sept. 2026.

Glover, Emily. "Home Security System Cost 2026." *Forbes Home*, Forbes Media, 28 Apr. 2026, www.forbes.com/home-improvement/home-security/home-security-system-cost/. Accessed 3 Sept. 2026.

"HC-SR04 Ultrasonic Range Finder Manual." *University of York*, www-users.cs.york.ac.uk/~mjf/roach/Datasheets/HCSR04_Manual.pdf. Accessed 3 Sept. 2026.

"Installing." *Arduino-ESP32 Documentation*, Espressif Systems, docs.espressif.com/projects/arduino-esp32/en/latest/installing.html. Accessed 3 Sept. 2026.

Joachimsmeyer, Armin, et al. *Arduino-IRremote*. GitHub, github.com/Arduino-IRremote/Arduino-IRremote. Accessed 3 Sept. 2026.

"Log Event." *Blynk Documentation*, Blynk, docs.blynk.io/en/blynk-library-firmware-api/log-event. Accessed 3 Sept. 2026.

"Manual Device Activation." *Blynk Documentation*, Blynk, docs.blynk.io/en/getting-started/activating-devices/manual-device-activation. Accessed 3 Sept. 2026.

"9V / 6F22 Battery." *PKCELL Battery*, www.pkcell.com/product/9v-6f22-battery/. Accessed 3 Sept. 2026.

uPesy. "ESP32 Wroom DevKit Pinout: Use the ESP32 GPIO Pins." *uPesy*, www.upesy.com/blogs/tutorials/esp32-pinout-reference-gpio-pins-ultimate-guide. Accessed 3 Sept. 2026.